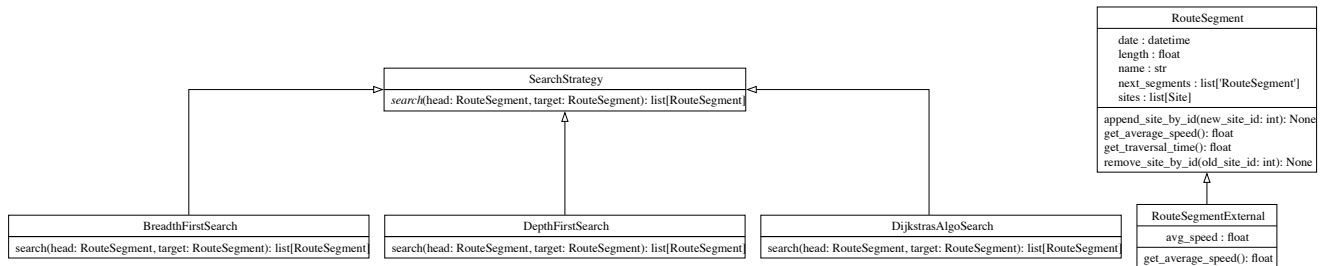


Fundamentals of CS II AE3

Classes and Layout



The classes used in `webtris_graph.py` are characterized by two main categories – search and routes.

Search Classes

`SearchStrategy` is an abstract class with one abstract method to implement, `search()`. The `search()` method is meant to be implemented by each subclass class as a different way to search through the given route. For input, each method takes a `head` (source) `RouteSegment` and target `RouteSegment`. Each subclass then implements the method to return an ordered list of `RouteSegments` with `head` at index 0 and `target` at index `len()-1`. `BreadthFirstSearch` uses a while loop to look for one of the paths with least nodes, prioritizing potential paths based on FIFO for its next segments. `DepthFirstSearch` calls an internal recursive method to search based on FIFO through its next segments for one valid path and returns the first one discovered. `DijkstrasAlgoSearch` searches using an implementation of Dijkstra's Algorithm to find the path taking the minimum amount of time.

RouteSegment

`RouteSegment` is a class which contains an edge and implied node on the graph representing all paths. It contains a `date` for which its data is considered valid, a `name`, a `length` of that segment in miles, a list of the next `RouteSegments` that are reachable from the end of this segment, and a list of `Site`s that are along the segment. Multiple segments in a list can represent a complete route, and information can be discerned of them from that list.

Modeling the possible routes graph based on edges rather than storing information in nodes was decided based on the context of this problem, where each edge has set information such as its `length` and contained `Site`s. The decision to not include a separate node class was also deliberate due to the nature of the problems presented. While nodes may improve performance in a dense possible route graph, this program is intended to be used for lightweight planning applications. For a more thorough route-planning system, a completely different representation of nodes and edges, such as representing sites as nodes, would be required and is outside the scope of this project.

RouteSegmentExternal

`RouteSegmentExternal` is a subclass of `RouteSegment` overriding the `get_average_speed()` method to return a constant given at initialization. These classes cannot hold sites. This is used for destinations, such as a zero-length `RouteSegmentExternal` to represent Heathrow, or a local roads segment with pre-defined length and traversal time.

Depth First Search (DFS)

Depth first search optimizes for finding a valid path as fast as possible. The implementation of DFS used in this project works by exploring every `RouteSegment`'s lowest-indexed (left) child from `next_segments`. As soon as it has found a single, valid path, it will return it. This does not guarantee that the path is in any way optimized, but rather that it is just a valid path. For the Gatwick -> Heathrow problem, my program found the A30 Path (Route C). For `2026-01-19T08:59:00`, it found this path would take 34.7 minutes. This result is different than the other algorithms because it is optimizing for runtime, not path traversal time or node count. This means that a very suboptimal path may be returned, as is the case in this situation as the path returned is entirely dependent on the order in which the `RouteSegment`s are constructed. If, for example, local roads were added to `m25_J12`'s `next_segments` before A30 was, then that segment instead would be returned.

Breadth First Search (BFS)

Breadth first search optimizes for the path with the fewest total nodes. The implementation of BFS I use in this project involves keeping track of a queue of potential paths and using a while exists loop to iterate upon it. For every iteration of the loop, the front path is popped and considered. If it contains `head` and `target`, it will be returned as the first path. Otherwise, each child of that element will be enqueued as a full potential path from the current point and the next iteration continues. This algorithm returned the local roads route (Route B) for the Gatwick -> Heathrow problem, with a total time of 47.8 minutes. Finding Route B makes logical sense, as it is the least nodes required to get to the target (two nodes versus three for Routes C and A). This result is different because, unlike depth first search, it optimizes for the shortest (by node count) path. Each level of the graph is considered before moving onto the next one, leading to a more thorough exploration of the graph.

Dijkstra's Algorithm

Dijkstra's Algorithm optimizes for a chosen factor. For this project, I chose to find the path taking the least amount of time as that's what would be my prioritizing factor while commuting. The algorithm operates similarly to BFS, instead using a priority queue to prioritize discovering paths taking the least amount of time and discover which are valid and have a low traversal time first. This lead to it discovering the most time-optimal route, taking the M25 (Route A). The fastest discovered so far, Route A can be traversed in 34.1 minutes. This result is different from the previous two because rather than optimize for the number of nodes, Dijkstra's is willing to go through more nodes if each connection between nodes has a lower weight (time to traverse).

Comparison and Differences

There are situations in which different algorithms are preferable to each other. In situations requiring high speed and simply need to know whether a path between two points exists or not may prefer to use depth first search. Its ability to find paths by deep discovery very early on lend to those use cases. One such use case may be seeing if two networks are connected. DFS may search through all of network A's devices to see if any of them also service network B for a user who needs to communicate between them. This is different from the use cases of breadth first search, which may be better suited for node-count optimization problems. A present use case for this algorithm is LinkedIn – which displays how closely users are connected. Using BFS to see what the closest level (1st, 2nd, 3rd, 3rd+) of connection a user has is a classic use case of BFS. Even if you have a 4th level connection with someone, you want to see that you're a 2nd level connection with them through someone else, hence the shortest path is needed. Dijkstra's Algorithm is tremendous at optimizing for weighted counts, such as paths. Each path in this problem has an associated weight, and being able to leverage that data to find the minimal amount of time needed to form a full route is crucial information to users. Even internal applications, such as routing network connections based on ping between each hop, can be incredibly important. Different algorithms can even be combined for a stronger product. If, for example, DFS is unable to find a road route from London to Paris, the application need not even run Dijkstra's, a very expensive task, on all roads in Europe. Additionally, if any path works for a given application and only some small percent of the time is an actual optimal path needed, this combination can prove efficient as well.

Conclusion

In this exercise, I would chose to use Dijkstra's algorithm for its ability to recognize the absolute shortest path for me, the commuter. In simple route choice questions such as these, compute power is hardly a problem and number of junctions is not relevant to users if junction density between paths requiring many road switches and staying on the same road is the same. In the end, the user just wants to get to the airport with the least amount of traffic. Test cases can be found in `test_webtris_graph.py`.